

TP1 - Application d'authentification forte MFA avec MongoDB, RBAC, ABAC et audit Big Data

Objectif général	Créer une application sécurisée qui authentifie les utilisateurs, impose un second facteur pour les données sensibles, applique des politiques RBAC/ABAC et journalise tous les accès dans MongoDB pour analyse.
Technologies	Python, Flask ou FastAPI, MongoDB, PyOTP/TOTP, JWT, bcrypt/passlib, pandas ou PySpark pour l'audit.

1. Scénario professionnel

Un hôpital souhaite sécuriser l'accès aux dossiers médicaux stockés dans une base NoSQL. Les médecins, infirmiers, secrétaires, patients et administrateurs sécurité n'ont pas les mêmes droits. Les données sensibles exigent une authentification multifacteur. Chaque tentative d'accès doit être enregistrée pour garantir la preuve, la traçabilité et la détection d'abus.

Architecture attendue

Utilisateur -> Interface Web/Login -> Authentification mot de passe + TOTP -> JWT -> Moteur RBAC/ABAC -> MongoDB resources -> Audit logs -> Notebook d'analyse.

2. Compétences visées

- Comprendre et implémenter une politique de contrôle d'accès formelle: sujet, objet, action, contexte.
- Mettre en œuvre le principe du moindre privilège avec RBAC.
- Ajouter des contraintes ABAC: service, sensibilité de la donnée, heure, adresse IP, statut MFA.
- Mettre en place une authentification double facteur avec TOTP.
- Produire des journaux d'audit exploitables par des technologies Big Data.

3. Données fournies

- datasets/users.csv: utilisateurs, rôles, départements, statut MFA.
- datasets/resources.csv: ressources, type, propriétaire, sensibilité.
- datasets/access_logs.csv: données de test pour le notebook d'audit.
- notebooks/TP1_MFA_RBAC_ABAC_MongoDB.ipynb: notebook de démarrage.

4. Travail demandé

Temps	Activité	Production attendue	Points de vigilance
0:00-0:25	Comprendre le scénario, créer la matrice RBAC/ABAC.	Tableau sujet-objet-action-contexte.	Ne pas donner un accès global admin à tous.
0:25-1:10	Créer la base MongoDB: users, resources, access_logs.	Collections initialisées et testées.	Stocker les mots de passe hachés, jamais en clair.
1:10-2:00	Développer /login avec mot de passe + génération du secret TOTP.	Connexion réussie pour un utilisateur MFA.	Le secret MFA doit être propre à chaque utilisateur.
2:00-2:40	Développer le moteur authorize(user, resource, action, context).	Décisions autoriser/refuser avec justification.	Tester au moins 8 cas, dont des refus.
2:40-3:20	Journaliser toutes les décisions dans access_logs.	Journal contenant: timestamp, user, action, ressource, décision, raison, IP.	L'audit doit tracer aussi les accès refusés.
3:20-4:00	Analyser les logs dans le	Top anomalies, utilisateurs	Relier les observations à

notebook et produire un à risque, recommandations. DICI et AAA.
mini rapport.

5. Exigences fonctionnelles minimales

1. Un formulaire de connexion ou une route API /login.
2. Un second facteur TOTP obligatoire si la ressource est confidentielle ou sensible.
3. Une politique RBAC pour les rôles medecin, infirmier, secretaire, admin_securite et patient.
4. Une contrainte ABAC de service: un médecin ou infirmier ne lit que les dossiers de son département.
5. Une contrainte horaire: refus hors plage de service, sauf admin_securite pour audit.
6. Un journal d'audit persistant dans MongoDB ou CSV si MongoDB indisponible.

6. Extrait de politique à implémenter

Rôle	Lire	Écrire	Créer	Exporter
Médecin	dossier médical de son service	dossier médical de son service	Non	Non
Infirmier	dossier médical de son service	Non	Non	Non
Secrétaire	dossier administratif	dossier administratif	dossier administratif	Non
Admin sécurité	journaux d'accès	Non	Non	journaux d'accès
Patient	son propre dossier administratif	Non	Non	Non

7. Pseudo-code de décision

```
def authorize(user, resource, action, context):
    if resource.sensitivity in ['confidentiel', 'sensible'] and not context.mfa_ok:
        return DENY, 'MFA obligatoire'
    if not rbac_allows(user.role, resource.type, action):
        return DENY, 'Rôle non autorisé'
    if user.role in ['medecin', 'infirmier'] and user.department != resource.owner_department:
        return DENY, 'Contrainte ABAC département'
    if context.hour < 6 or context.hour > 20:
        return DENY, 'Hors plage de service'
    return ALLOW, 'Accès autorisé'
```

8. Questions de réflexion

- Pourquoi la MFA est-elle plus importante pour les données sensibles ?
- Quelle différence faites-vous entre RBAC et ABAC dans ce TP ?
- Quels champs du journal permettent d'assurer la preuve et l'imputabilité ?
- Quels risques apparaissent si les accès refusés ne sont pas journalisés ?